Custom Browsers (FileExplorerAssistant Component)

Jul 13, 2022 • 7 minutes to read

IN THIS ARTICLE

Default Folder Browser

Initial Extension Setup

TreeList Extension

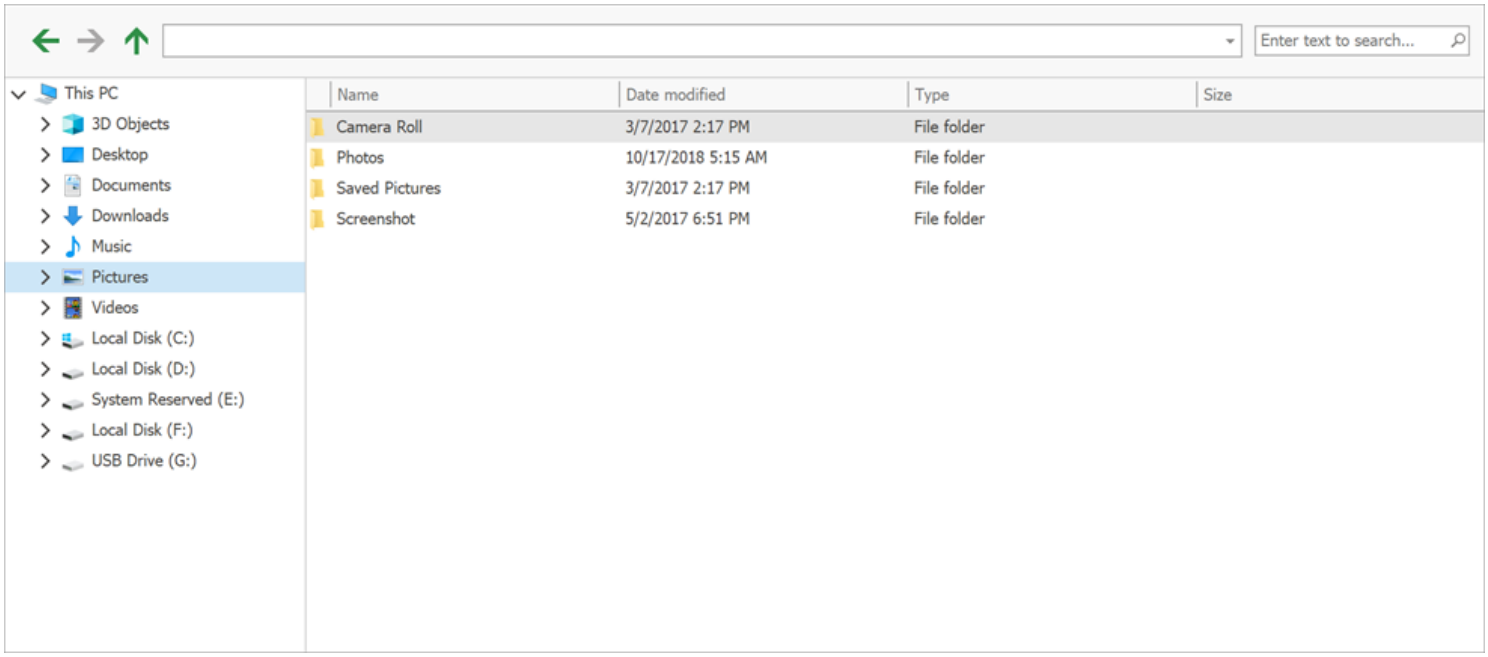
GridControl Extension

BreadcrumbEdit Extension

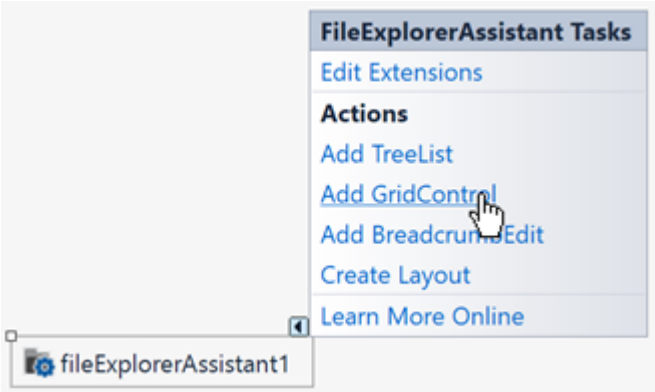
The DevExpress WinForms suite ships with skinnable counterparts for default WinForms file/folder dialogs: `XtraFolderBrowserDialog`, `XtraOpenFileDialog`, and `XtraSaveFileDialog`. The `FileExplorerAssistant` component allows you to embed similar functionality in a form or UserControl, and build custom file/folder explorers that can navigate local storage and/or virtual node hierarchy.

Default Folder Browser

This section explains core `FileExplorerAssistant` concepts and demonstrates how to build the standard file/folder browser similar to Windows Explorer.



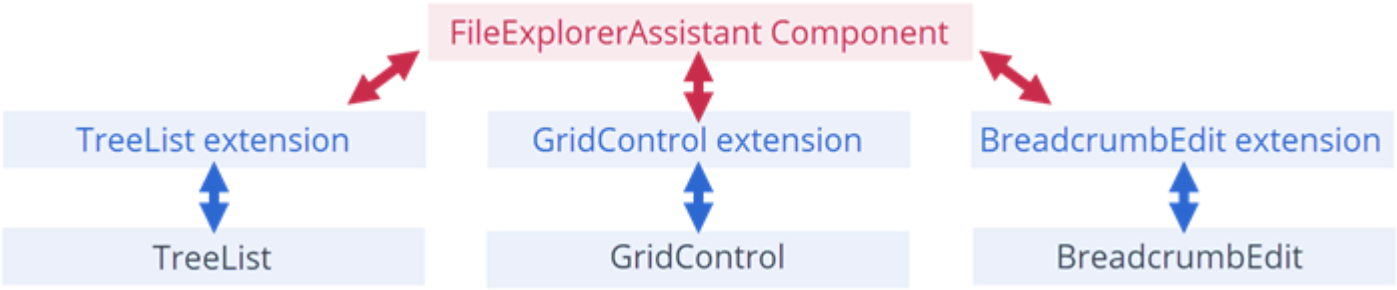
Place the `FileExplorerAssistant` component on a form or UserControl, and invoke its smart-tag menu.



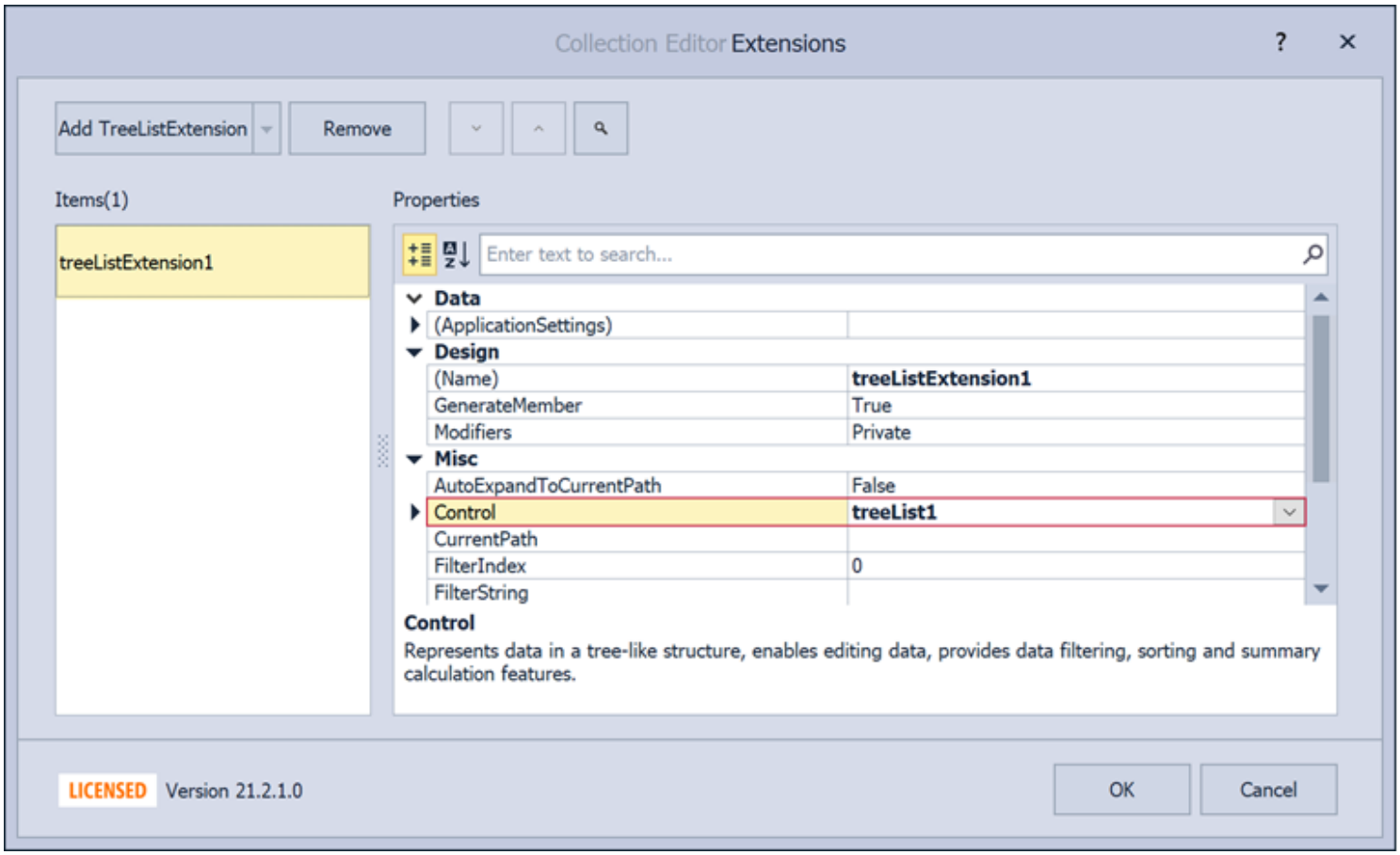
Click the “Add...” actions to add all three browser components:

- Side navigation menu (the `TreeList` control).
- Main client area (the `GridControl` control).
- Address bar (the `BreadCrumbEdit` control).

All parts are default DevExpress controls with FileExplorerAssistant **extensions** attached to them. Extensions customize the appearance and behavior of controls, and allow these controls to communicate with their parent FileExplorerAssistant component.



When you click an “Add...” smart-tag menu action, the component creates the corresponding control-extension pair. If needed, you can click the “Edit Extensions” smart-tag menu action to create standalone extensions and bind them to controls on a form or UserControl.

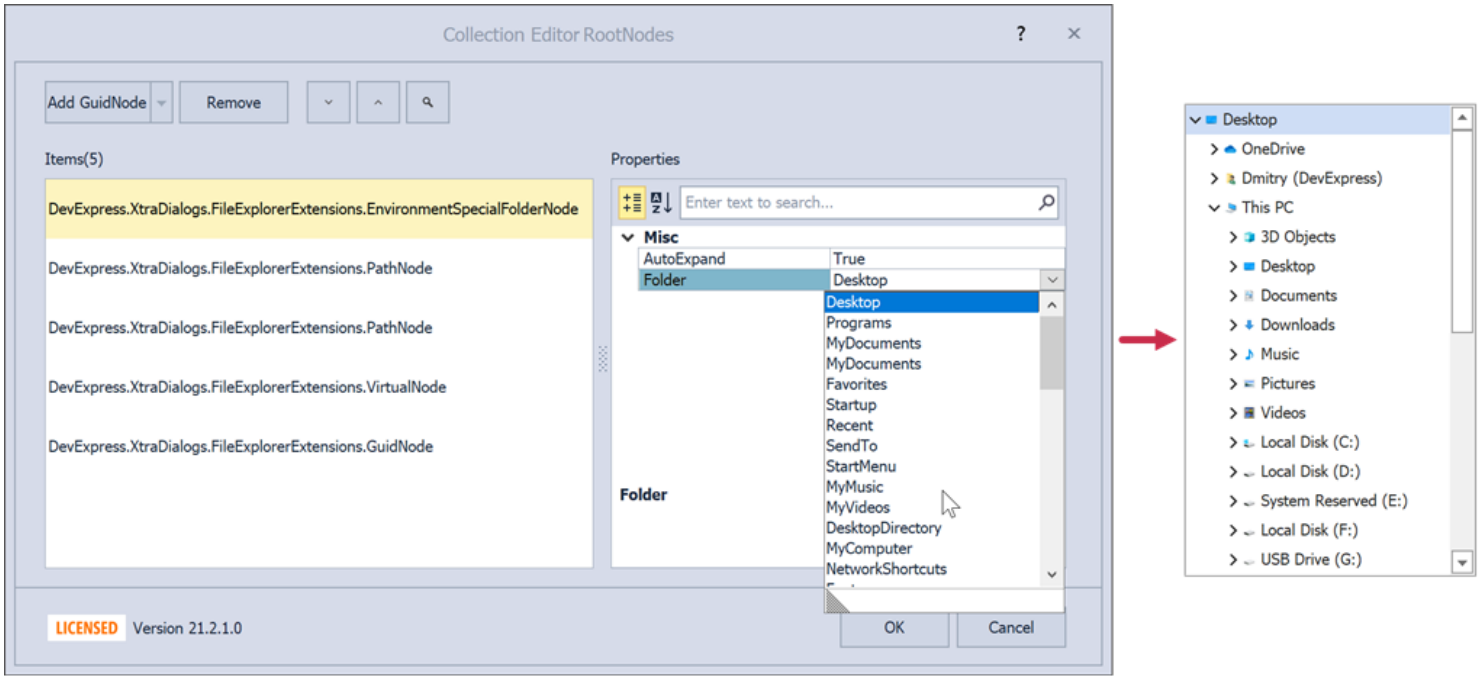


Initial Extension Setup

TreeList Extension

The TreeList Extension has the `RootNodes` collection that specifies the list of top-level nodes. Initially, this collection has only one node — the **EnvironmentSpecialFolderNode** that allows users to browse one of the special Windows folders↗: Desktop, My Documents, Program Files, Fonts, and others. The

initial value of this node is “Desktop”, which allows users to browse the entire local storage of the current PC.



You can modify this collection to add other nodes:

- **PathNode** points to a single local storage folder or drive. The path to this object is stored in the node’s `Path` property.

  C#

  ```
  // Scan for all partitions
  System.IO.DriveInfo[] driveList = System.IO.DriveInfo.GetDrives();
  foreach (var drive in driveList) {
      // Select only logical fixed partitions
      if (drive.DriveType == System.IO.DriveType.Fixed && drive.IsReady)
          // Add each drive as a root node
          treeListExtension1.RootNodes.Add(new PathNode(drive.RootDirectory.ToStrin
  }
  ```

- **GuidNode** allows you to add special folders that have GUIDs↗.

  C#

  ```
  // Add the "Network" special folder
  GuidNode nodeNetwork = new GuidNode(new Guid("d20beec4-5ca8-4905-ae3b-bf251ea09b5
  ```

```
nodeNetwork.AutoExpand = false;
treeListExtension1.RootNodes.Add(nodeNetwork);
```

- **VirtualFolderNode** and **VirtualItemNode** are custom nodes. You can specify their title and image, and VirtualFolderNodes can be populated with child nodes. Use these nodes to create custom entries that are not present on local storage.

GridControl Extension

The two core settings of the GridControl extension are:

- **ViewMode** — allows you to choose between eight Explorer styles (Tiles, Content, Large Icons, and others).
- **CurrentPath** — the currently browsed directory.

BreadcrumbEdit Extension

- **CurrentPath** is the currently browsed directory.

Connect Extensions

Extensions do not communicate with each other. For example, if a user selects a node inside a TreeList, the Data Grid does not reflect this navigation. You need to implement the communication between extensions based on the structure of your custom folder explorer.

Navigation Bindings

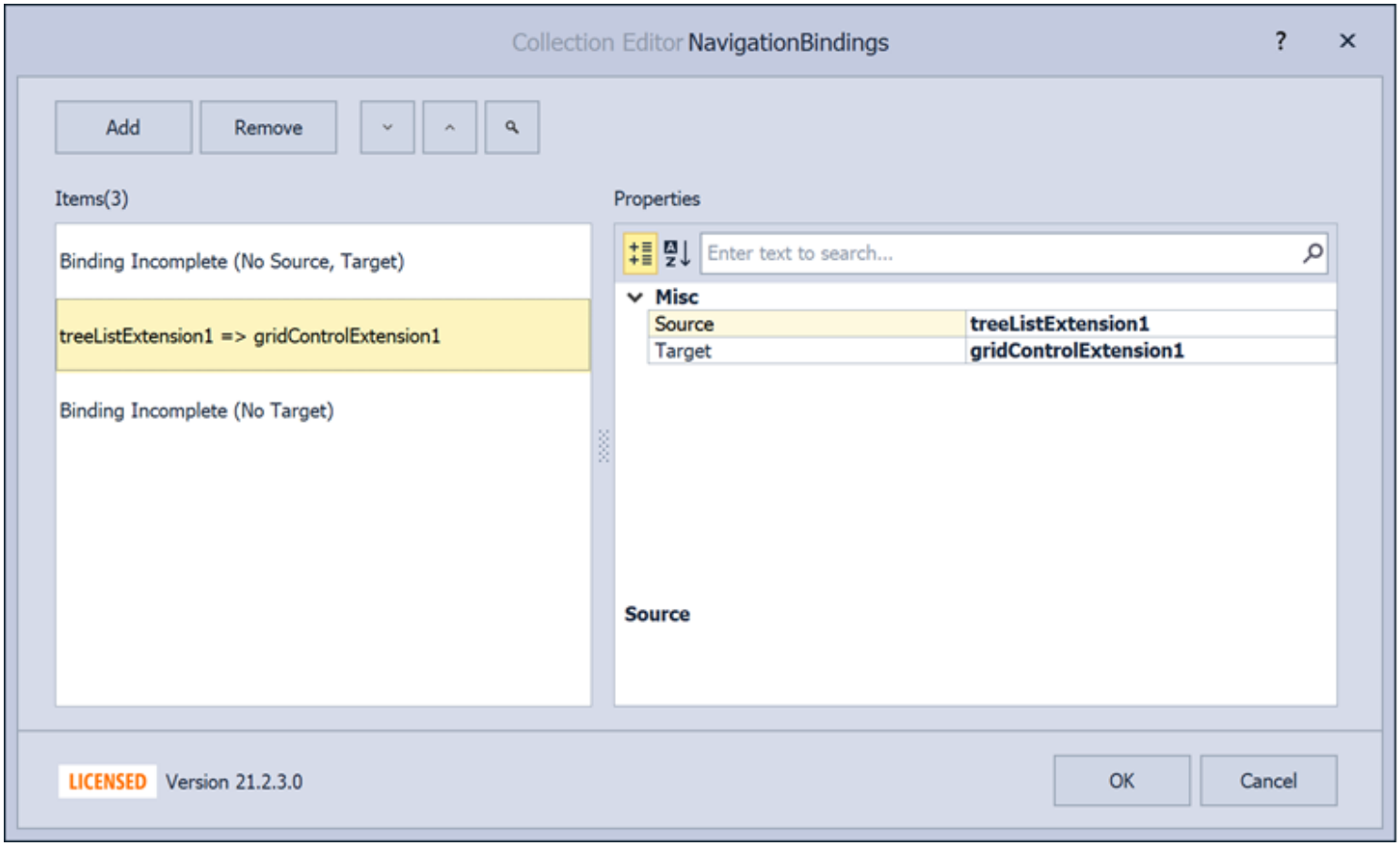
A "navigation binding" is an object of the `DevExpress.XtraDialogs.NavigationBinding` class. These objects have two properties:

- `Source` — specifies a control whose navigation is tracked.
- `Target` — specifies a control that should replicate the `Source` control navigation.

When stored inside the `FileExplorerAssistant.NavigationBindings` collection, navigation bindings transmit navigation actions from one extension owned by this FileExplorerAssistant component to another.

You can add navigation bindings at design time (click the ellipsis button next to the "NavigationBindings" property in the Visual Studio Property Grid) or in code. At design time, the

Collection Editor dialog checks whether both `Target` and `Source` properties are set, and highlights bindings that are missing either of these settings.



You can also edit navigation bindings in code:

C#

```
NavigationBinding nbGridToBreadcrumb = new NavigationBinding();
nbGridToBreadcrumb.Source = gridControlExtension1;
nbGridToBreadcrumb.Target = breadCrumbExtension1;
fileExplorerAssistant1.NavigationBindings.Add(nbGridToBreadcrumb);
```

Navigation Events

Another way to connect extensions is to handle the `CurrentItemChanged` event available for every FileExplorerAssistant extension. This event fires every time a user selects a different item within a control. Handle `CurrentItemChanged` if you need to implement a custom extension communication logic that cannot be implemented with navigation bindings.

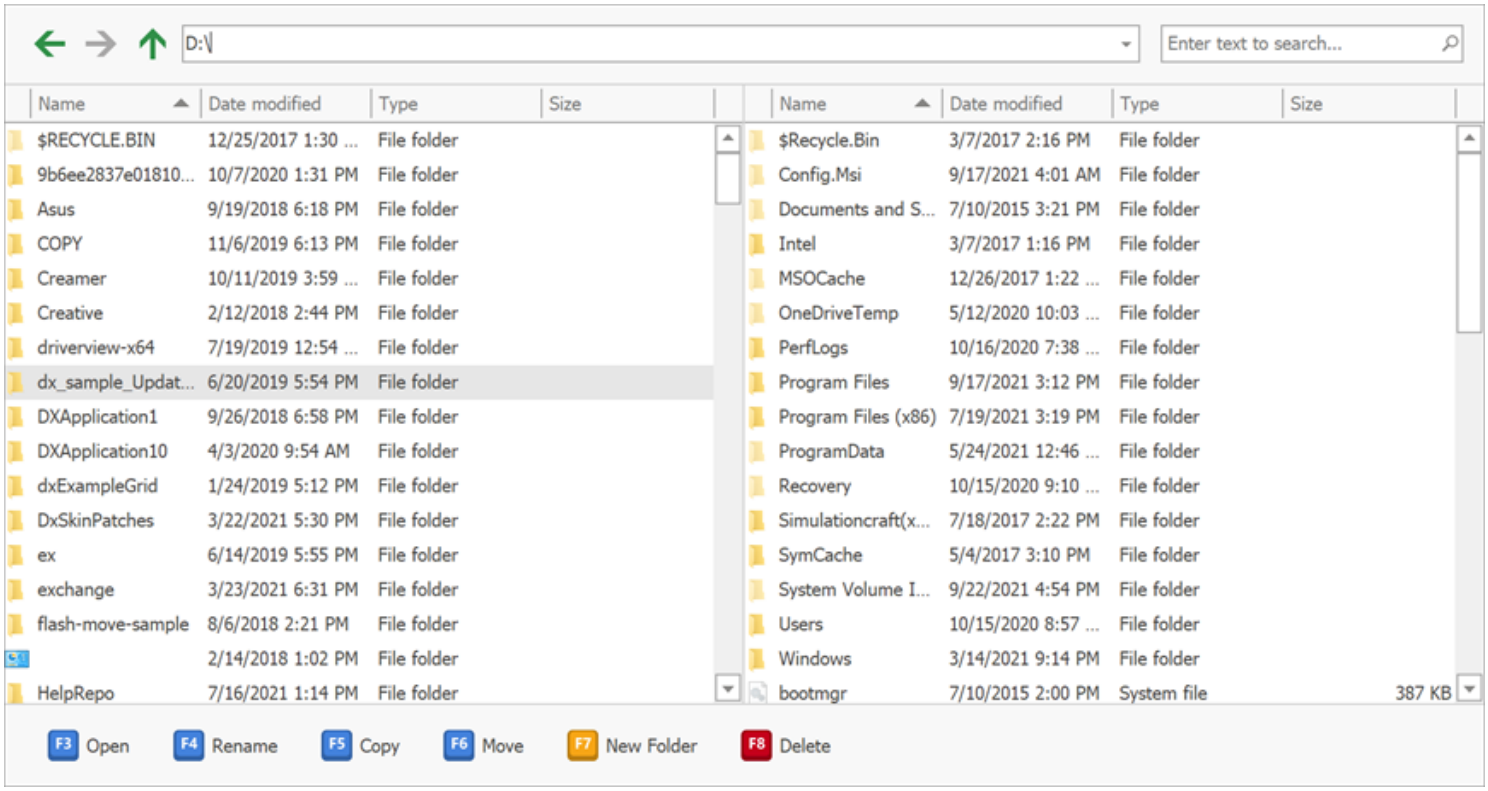
C#

```
private void treeListExtension1_CurrentItemChanged(object sender, CurrentItemChangedEv
    // Update Data Grid and Breadcrumb when the Tree List selection changes
    gridControlExtension1.SetCurrentItem(e.CurrentItem);
    breadCrumbExtension1.SetCurrentItem(e.CurrentItem);
}
```

Navigation and Edit Buttons

FileExplorerAssistant extensions expose an API that allows you to manage files and folders. This section demonstrates how to use this API to implement action buttons similar to those seen in the “Two-Panel File Manager” DevExpress demo.

Run Demo: Two-Panel File Manager, XtraDialogs Demo



- `DoDefaultAction` — navigates inside the selected folder, or opens the selected file in a default program associated with the file’s extension (Notepad for .txt files, Photos for .png files, and so on).
- `Rename` — renames the selected file or folder.
- `CopySelectedItem` and `MoveSelectedItem` — copy or move the selected file or folder to the location specified by the method’s parameter. In the code below the selected file is copied to the currently open folder of another Grid.

C#

```
gridControlExtension1.CopySelectedItem(gridControlExtension2.CurrentFolder);
```

- `CreateNewFolder` — creates a new directory inside the currently browsed folder.
- `GoBack`, `GoForward`, `GoUp` — navigation methods.

C#

```
// Navigate to the parent folder
gridControlExtension1.GoUp();
```

- Properties that start with “Can” return whether the corresponding action can be executed for the current folder or file. For instance, if a user browses the SpecialEnvironmentFolder.Desktop, they cannot navigate to the parent folder (the `GoUp` method is not available), and the `CanGoUp` property returns **false**.

C#

```
// Set the availability of a button that calls the Rename method
btnRename.Enabled = gridControlExtension1.CanRename;
```

- `SetCurrentPath` and `SetCurrentSpecialFolder` — allow you to browse a specific folder or special Windows folder.

C#

```
breadCrumbExtension1.SetCurrentSpecialFolder(Environment.SpecialFolder.CommonStar
// or
breadCrumbExtension1.SetCurrentPath(
    Environment.GetFolderPath(Environment.SpecialFolder.CommonStartMenu));
```

Custom Hierarchy

The **FileExplorerAssistant** component allows you to create solutions that browse not only folders on local storage, but also virtual (custom) hierarchies.



Run Demo: Car Rental Catalogue, XtraDialogs Demo

To create a custom hierarchy for the Tree List control, populate the `RootNodes` collection of the control's related extension with `VirtualFolderNode` and `VirtualItemNode` instances.

VirtualFolderNode class objects serve as "folders". These objects have the `Nodes` collection that you can use to populate them with child folders or items.

C#

```
VirtualFolderNode rootNode = new VirtualFolderNode("Root Node");
VirtualFolderNode childNode = new VirtualFolderNode("Child Node 1");
rootNode.ImageOptions.SvgImage = svgImageCollection1[0];
childNode.ImageOptions.SvgImage = svgImageCollection1[1];
// ...
// Add more child nodes
rootNode.Nodes.Add(childNode);
treeListExtension1.RootNodes.Add(rootNode);
```

VirtualItemNode class objects serve as "files". These objects cannot embed other folders or items, and are not shown inside a Tree List (similar to local storage files).

C#

```
VirtualFolderNode rootNode = new VirtualFolderNode("Root Node");
rootNode.Nodes.Add(new VirtualFolderNode("Child Node 1"));
((VirtualFolderNode)rootNode.Nodes[0]).Nodes.Add(item1);
```

Combine virtual "folder" and "file" objects to create a custom hierarchy as your needs dictate.

Use of this site constitutes acceptance of our Website Terms of Use and Privacy Policy
Copyright © 1998-2022 Developer Express Inc. All trademarks or registered trademarks are property of their respective owners